

机器学习 lab 2 Report

PB22111599 杨映川

1#代码实现

对数据实现划分

- 使用 `train_test_split` 函数直接进行划分

```
1 X_train, X_test, Y_train, Y_test = train_test_split(X, Y, test_size=0.2,
  random_state=42)
```

构建mlp模型

- 直接使用 `MLPClassifier` 函数, 并梳理其参数的含义

```
1 """
2 ? hidden_layer_sizes 隐藏层有几个neurons
3 ? activation: {'identity', 'logistic', 'tanh', 'relu'}, default='relu'
4 ? solver: {'lbfgs', 'sgd', 'adam'}, default='adam'
5 ? alpha 正则化参数, default=0.0001
6 ? learning_rate: {'constant', 'invscaling', 'adaptive'}, default='constant'
7 ? learning_rate_init , default=0.001
8 ? max_iter 最大迭代次数
9 ? shuffle 是否在每次迭代时打乱数据, default=True
10 ? random_state 随机种子
11 ? tol 优化的容忍度, default=1e-4
12 """
13 mlp = MLPClassifier(hidden_layer_sizes=(10,),
14                      activation='tanh', # modified
15                      solver='adam',
16                      alpha=0.0001,
17                      max_iter=1000, # modified
18                      random_state=42)
```

拟合数据、预测结果

```
1 mlp.fit(standard_train, Y_train)
2 result = mlp.predict(standard_test)
```

神经网络内部函数

- sigmoid函数和其导数函数直接计算
- 前向传播: 将每一层的输入的计算结果的激活值逐层传递, 直到生成最终结果, $t = f(\vec{W}'\vec{A} + b)$
- 反向传播: 分别计算输出层和隐藏层的误差error与梯度delta, 然后使用梯度下降的方法更新权重与偏置, 逐步调整神经网络。

```

1 def sigmoid(self, x): # sigmoid 计算方式
2     """
3     TODO在此填入你的代码
4     """
5     return 1 / (1 + np.exp(-x))
6
7 def sigmoid_derivative(self, x): # sigmoid 导数计算方式
8     """
9     TODO在此填入你的代码
10    """
11    return x * (1 - x)
12
13 def forward(self, x):
14    """
15    TODO在此填入你的代码
16    """
17    # input -> hidden
18    self.hidden_input = x @ self.weights_input_hidden + self.bias_hidden
19    self.hidden_output = self.sigmoid(self.hidden_input)
20    # hidden -> output
21    self.output_input = self.hidden_output @ self.weights_hidden_output +
self.bias_output
22    self.output = self.sigmoid(self.output_input)
23    return self.output
24
25 def backward(self, x, y, output, learning_rate):
26    """
27    TODO在此填入你的代码
28    """
29    output_error = y - output
30    output_delta = output_error * self.sigmoid_derivative(output)
31
32    hidden_error = output_delta @ self.weights_hidden_output.T
33    hidden_delta = hidden_error *
self.sigmoid_derivative(self.hidden_output)
34
35    # update the weights and biases
36    self.weights_hidden_output += (self.hidden_output.T @ output_delta) *
learning_rate
37    self.bias_output += np.sum(output_delta, axis=0, keepdims=True) *
learning_rate
38    self.weights_input_hidden += (x.T @ hidden_delta) * learning_rate
39    self.bias_hidden += np.sum(hidden_delta, axis=0, keepdims=True) *
learning_rate
40

```

2#结果分析

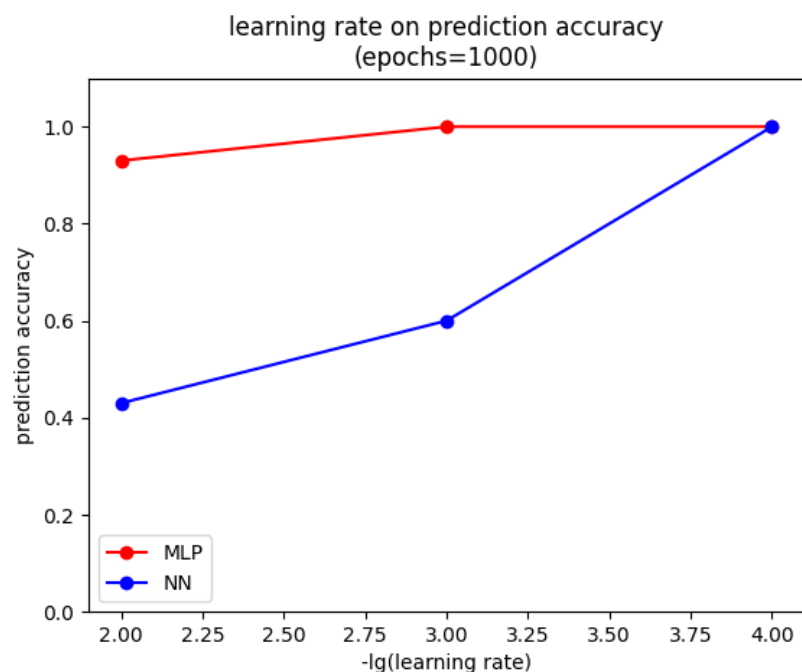
1. 当隐藏层尺寸为10，表中数据第一个为自带的MLPClassifier的预测准确率，第二个为手动实现的神经网络的预测准确率，单位均为%

hidden_size=10	learning_rate	learning_rate	learning_rate
epochs	=0.0001	=0.001	=0.01
10	0.03, 0.47	0.10, 0.40	0.67, 0.57
100	0.10, 0.60	0.67, 0.50	1.00 , 0.93
1000	0.70, 0.57	1.00 , 0.77	1.00, 0.97

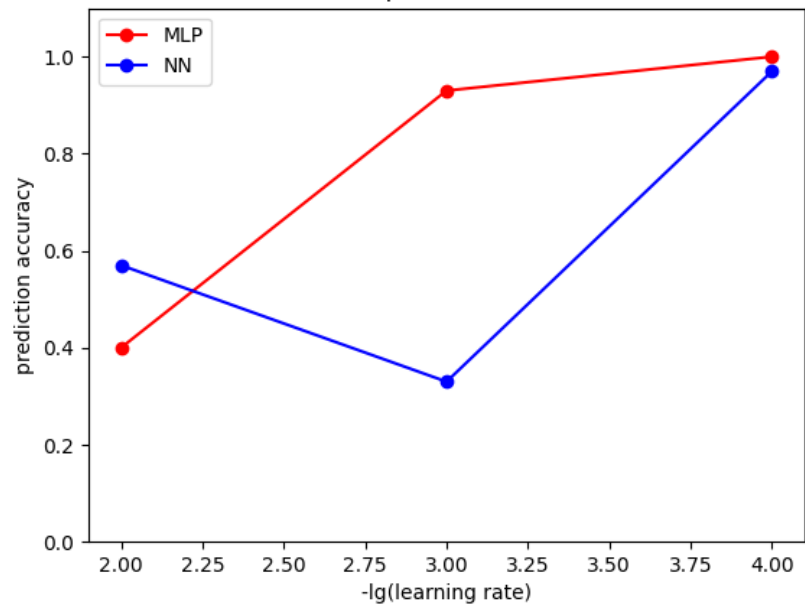
2. 当隐藏层尺寸为100，表中数据第一个为自带的MLPClassifier的预测准确率，第二个为手动实现的神经网络的预测准确率，单位均为%

hidden_size=100	learning_rate	learning_rate	learning_rate
epochs	=0.0001	=0.001	=0.01
10	0.33, 0.00	0.40, 0.20	0.90, 0.63
100	0.40, 0.57	0.93, 0.33	1.00 , 0.97
1000	0.93, 0.43	1.00 , 0.60	1.00, 1.00

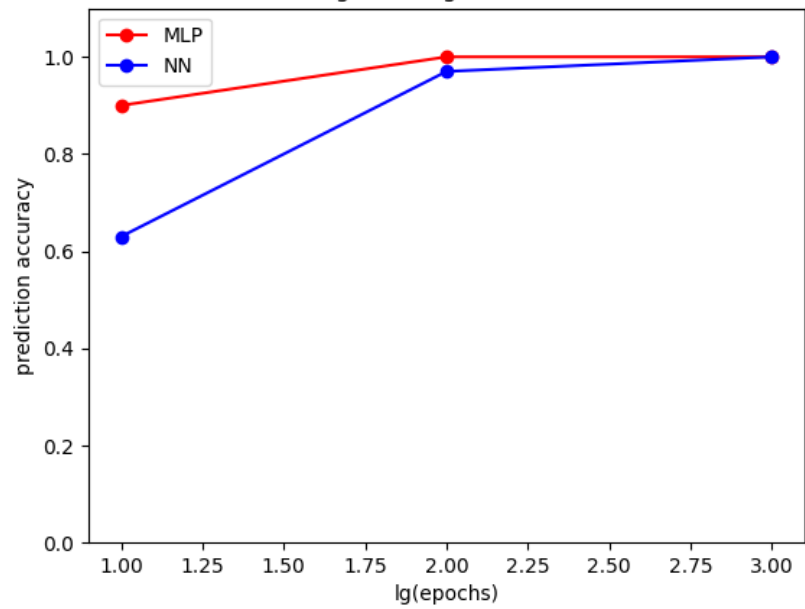
3. 我们选取第二个表格（hidden_size=100）进行可视化。这四个图中，前两个是epochs确定时学习率对准确率的影响折线图；后两个是学习率确定时epochs对准确率的影响折线图。

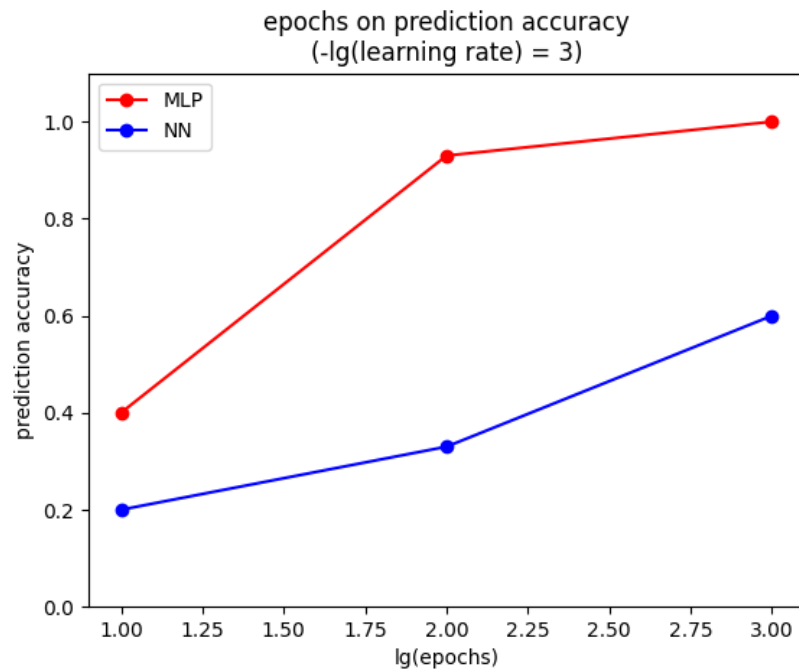


learning rate on prediction accuracy
(epochs=100)



epochs on prediction accuracy
($-\lg(\text{learning rate}) = 2$)





4. 不管是从表格数据还是图表数据都能看出，在参数设定相同的情况下，MLClassifier的预测准确率整体要比手动实现的神经网络更好也更稳定。
5. 由于数据数量较少且特征较为明显，两个模型在选定比较好的参数后都能达到100%的预测准确率。

3#其他

- 本次实验耗时
 - 代码补全1h
 - 查阅资料1h
 - 参数调试与报告撰写2h
- 反馈：暂无